

Question 1

A supplier drops delivery files into a Unity Catalog volume at unpredictable times, sometimes twice an hour, sometimes not for two days. Processing should begin shortly after each drop, with no runs on empty days.

Which trigger should the job use?

- A. A table update trigger on the bronze table, which then starts the job when the supplier's new files are appended to that bronze table's data
- B. A scheduled trigger every 15 minutes, with a first task that checks the volume path and ends the run early whenever no new files are present there
- C. A file arrival trigger on the volume path, which monitors that location and then starts a run whenever it detects newly added files there
- D. A continuous trigger, which keeps one run active at all times and processes any new files that then land in the volume while that run is active

Correct answer: C

C. A file arrival trigger on the volume path, which monitors that location and then starts a run whenever it detects newly added files there

Explanation

Correct answer:

A file arrival trigger on the volume path, which monitors that location and then starts a run whenever it detects newly added files there

Why this is correct:

A file-arrival trigger monitors files under a Unity Catalog volume path and starts the job when new files appear. This trigger is designed for irregular arrivals, so it avoids both fixed polling runs and executions on days with no deliveries.

Why the other options are wrong:

A scheduled trigger every 15 minutes, with a first task that checks the volume path and ends the run early whenever no new files are present there: A frequent schedule launches whether or not a supplier has uploaded anything, and the first task must repeatedly inspect the path. The stem asks for event-driven runs with no empty-day executions.

A continuous trigger, which keeps one run active at all times and processes any new files that then land in the volume while that run is active: A continuous trigger is for jobs that should run continuously and restart runs as they complete. It does not wait specifically for a new file event.

A table update trigger on the bronze table, which then starts the job when the supplier's new files

are appended to that bronze table's data: The supplier has not caused a table commit yet; the job needs to respond to the earlier file landing event in the volume.

References:

1. Trigger jobs when new files arrive: <https://docs.databricks.com/aws/en/jobs/file-arrival-triggers>

Question 2

A bronze column named `quantity` arrives as `STRING`. Valid integer text must become `INT` and malformed values `NULL`, with every row kept for later review. ANSI mode is on, and the query must not fail on bad input.

Which expression meets these requirements?

- A. `CAST(quantity AS INT)`
- B. `COALESCE(TRY_CAST(quantity AS INT), 0)`
- C. `TRY_CAST(quantity AS INT)`
- D. `TRY_CAST(quantity AS DECIMAL(10, 0))`

Correct answer: C

C. `TRY_CAST(quantity AS INT)`

Explanation

Correct answer:

`TRY_CAST(quantity AS INT)`

Why this is correct:

`TRY_CAST` converts valid integer strings to `INT` and returns `NULL` for malformed values instead of throwing an ANSI-mode cast error. It is a projection expression, so it preserves every input row for review while making invalid quantities visible as `NULL`.

Why the other options are wrong:

`COALESCE(TRY_CAST(quantity AS INT), 0)`: Replacing failed conversions with zero erases the distinction between corrupt input and a genuine quantity of zero. The requirement explicitly calls for `NULL` as the review signal.

`CAST(quantity AS INT)`: `CAST` raises an error for malformed strings under ANSI mode, aborting the query instead of retaining the row with a `NULL` result.

`TRY_CAST(quantity AS DECIMAL(10, 0))`: A decimal target does not satisfy the requested `INT` type. It also does not provide the specified output schema, even though `TRY_CAST` could tolerate malformed input.

References:

1. **cast function:** <https://docs.databricks.com/aws/en/sql/language-manual/functions/cast>

2. **CAST_INVALID_INPUT error condition:**

<https://docs.databricks.com/aws/en/error-messages/cast-invalid-input-error-class>

Question 3

A join stage runs 240 tasks. Most finish in about 12 seconds after reading roughly 80 MB of shuffle data, but three each read over 3 GB and run for minutes.

Which problem do these metrics most strongly indicate?

- A. Too low a `spark.sql.shuffle.partitions` setting, where every task in the stage handles too much data and then spills part of it out to disk
- B. Data skew, because a few key values carry so many of the rows that their partitions end up far larger than all the other partitions produced there
- C. Straggler executors on an unhealthy node, where its slow hardware makes a few tasks run minutes longer than other tasks processing similar data
- D. Unsplittable input files, where each of those three tasks had to read a whole compressed file in one piece because the format cannot be split

Correct answer: B

B. Data skew, because a few key values carry so many of the rows that their partitions end up far larger than all the other partitions produced there

Explanation

Correct answer:

Data skew, because a few key values carry so many of the rows that their partitions end up far larger than all the other partitions produced there

Why this is correct:

The task data volumes are extremely uneven: three tasks read over 3 GB while the rest read about 80 MB. That pattern points to skew, where a small number of key values concentrate rows into a few oversized shuffle partitions and keep those tasks running long after the others finish.

Why the other options are wrong:

Straggler executors on an unhealthy node, where its slow hardware makes a few tasks run minutes longer than other tasks processing similar data: Hardware delays affect execution time, but cannot explain why only those tasks read dramatically more data; the shuffle partitions themselves are imbalanced.

Unsplittable input files, where each of those three tasks had to read a whole compressed file in one piece because the format cannot be split: Unsplittable input files can limit parallelism during file reads. These measurements are from a join's shuffle stage, where the standout symptom is a few much larger partitions.

Too low a `spark.sql.shuffle.partitions` setting, where every task in the stage handles too much

data and then spills part of it out to disk: Too few shuffle partitions would tend to make task sizes broadly large. Here almost every task is small and only three are extreme, which is the signature of skew.

References:

1. Skew and spill:

<https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-page>

Question 4

Retransmissions duplicate rows of readings per device_id and reading_time, and copies can share a received_at value. Exactly one copy with the latest received_at must be kept.

```
w = Window.partitionBy("device_id",  
"reading_time").orderBy(F.col("received_at").desc())
```

Which expression meets this requirement?

- A. `readings.withColumn("rn", F.rank().over(w)).filter("rn = 1").drop("rn")`
- B. `readings.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")`
- C. `readings.orderBy(F.col("received_at").desc()).dropDuplicates(["device_id", "reading_time"])`
- D. `readings.withColumn("rn", F.dense_rank().over(w)).filter("rn = 1").drop("rn")`

Correct answer: B

B. `readings.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")`

Explanation

Correct answer:

```
readings.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

Why this is correct:

row_number assigns exactly one row number 1 within each (device_id, reading_time) partition, including when timestamps tie. Ordering by descending received_at puts a latest-time copy first; filtering to 1 keeps one copy rather than every tied rank.

Why the other options are wrong:

readings.orderBy(F.col("received_at").desc()).dropDuplicates(["device_id", "reading_time"]): dropDuplicates does not guarantee that the retained copy is the one with the latest received_at; a prior global sort is not a reliable per-key winner rule.

readings.withColumn("rn", F.rank().over(w)).filter("rn = 1").drop("rn"): rank gives the same rank to tied latest timestamps, so filtering for rank 1 can keep multiple copies rather than exactly one.

readings.withColumn("rn", F.dense_rank().over(w)).filter("rn = 1").drop("rn"): dense_rank also assigns rank 1 to every row tied at the maximum timestamp, violating the requirement to retain a single

row.

References:

1. Window functions:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-window-functions>

Question 5

A Lakeflow Job accepts a job parameter named `load_mode`. When its value is full, a full-refresh task must run; for any other value, an incremental task must run instead, and the run should still succeed.

How should the branching be configured?

- A. A For each task over `load_mode`'s value, whose nested task then runs the full refresh when its input equals full and runs the incremental load otherwise
- B. A notebook task that first raises an error unless `load_mode` equals full, with the incremental task using the Run if condition At least one failed
- C. An If/else task that compares `load_mode` with full, with the full-refresh task depending on that task and the incremental task using Run if All failed
- D. An If/else task comparing `load_mode` with full, with the full refresh task depending on its true outcome and the incremental task on false

Correct answer: D

D. An If/else task comparing `load_mode` with full, with the full refresh task depending on its true outcome and the incremental task on false

Explanation

Correct answer:

An If/else task comparing `load_mode` with full, with the full refresh task depending on its true outcome and the incremental task on false

Why this is correct:

An If/else condition task evaluates a boolean expression using the job parameter, and dependent tasks can be attached to its true and false outcomes. Comparing `{{job.parameters.load_mode}}` with full routes each run to exactly the appropriate refresh path.

Why the other options are wrong:

An If/else task that compares `load_mode` with full, with the full-refresh task depending on that task and the incremental task using Run if All failed: A Run if condition checks upstream task result states such as success or failure, not the contents of a job parameter.

A notebook task that first raises an error unless `load_mode` equals full, with the incremental task using the Run if condition At least one failed: A For each task repeats work for items in an input array. It does not select one of two mutually exclusive tasks based on a string parameter.

A For each task over `load_mode`'s value, whose nested task then runs the full refresh when its input equals full and runs the incremental load otherwise: A task value is produced by an upstream

task during a run, while `load_mode` is already supplied as a job parameter. Branching directly on that parameter is simpler and matches the stated input.

References:

1. Add branching logic to a job with the If/else task:

<https://docs.databricks.com/gcp/en/jobs/tasks/if-else>

Question 6

A notebook task in a daily job sometimes fails because an upstream service is briefly unavailable, and rerunning it is safe. The team wants another attempt after a short pause, and the job must keep its daily schedule.

Which setting meets this need?

- A. A copy of the task that depends on it with its Run if set to At least one failed, so the copy runs once right after any failure of the original task
- B. A shorter timeout on the notebook task with retries left set at zero, so the failing attempt ends sooner and frees up its compute for the next run
- C. Task retries with a minimum retry interval, so the notebook reruns automatically once that interval passes, up to the configured maximum retry count
- D. A continuous trigger on the job, which restarts the failed runs automatically with exponential backoff between the attempts it makes

Correct answer: C

C. Task retries with a minimum retry interval, so the notebook reruns automatically once that interval passes, up to the configured maximum retry count

Explanation

Correct answer:

Task retries with a minimum retry interval, so the notebook reruns automatically once that interval passes, up to the configured maximum retry count

Why this is correct:

Task retry settings let Lakeflow Jobs automatically retry a failed task after a configured minimum interval and up to a maximum retry count. Since the failure is transient and rerunning is safe, this retries the notebook without changing the daily schedule.

Why the other options are wrong:

A copy of the task that depends on it with its Run if set to At least one failed, so the copy runs once right after any failure of the original task: A dependent alert or copy task would be a separate DAG node; it does not rerun the original notebook after a delay.

A continuous trigger on the job, which restarts the failed runs automatically with exponential backoff between the attempts it makes: This trigger replaces the daily cadence with repeat-on-completion behavior; it does not configure another attempt for one failed task.

A shorter timeout on the notebook task with retries left set at zero, so the failing attempt ends sooner and frees up its compute for the next run: Ending an attempt sooner does not cause another

attempt when the retry count is zero.

References:

1. **Jobs API 2.0:** <https://docs.databricks.com/aws/en/reference/jobs-2.0-api>

Question 7

The engineers group holds **SELECT** and **MODIFY** on the **ops.silver** schema and has no direct grants on **ops.silver.assets**. It must stop changing that one table's data but keep **MODIFY** on the schema's other existing tables.

Which approach meets the requirement?

- A. Run **DENY MODIFY** on **ops.silver.assets** for engineers, which then overrides the schema-level grant while the other tables keep their own access
- B. Revoke **MODIFY** on the schema, then grant **MODIFY** again on each other table, since a table-level revoke cannot remove that inherited grant
- C. Transfer ownership of **ops.silver.assets** to another group, which then removes every privilege that engineers inherit on that one table
- D. Revoke **MODIFY** on **ops.silver.assets** from engineers, which then blocks the privilege inherited from the schema for that single table only

Correct answer: B

B. Revoke **MODIFY** on the schema, then grant **MODIFY** again on each other table, since a table-level revoke cannot remove that inherited grant

Explanation

Correct answer:

Revoke **MODIFY** on the schema, then grant **MODIFY** again on each other table, since a table-level revoke cannot remove that inherited grant

Why this is correct:

A schema-level **MODIFY** grant is inherited by its tables, and a child table cannot cancel that inherited access with a direct revoke. Remove the schema grant, then grant **MODIFY** to the group on the other tables where it should remain.

Why the other options are wrong:

Revoke **MODIFY on **ops.silver.assets** from engineers, which then blocks the privilege inherited from the schema for that single table only:** Revoking **MODIFY** directly on assets cannot remove the privilege inherited from the schema. The inherited grant must be removed at its source.

Run **DENY MODIFY on **ops.silver.assets** for engineers, which then overrides the schema-level grant while the other tables keep their own access:** Revoking **SELECT** on the table addresses reading, not the group's **MODIFY** capability. It also leaves the schema-level grant that provides write access.

Transfer ownership of **ops.silver.assets to another group, which then removes every privilege**

that engineers inherit on that one table: Transferring ownership is unnecessary and still leaves the inherited schema privilege in place. Ownership is broader than the least-privilege change the question needs.

References:

1. Unity Catalog permissions model concepts:

<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/access-control/permissions-concepts>

Question 8

A reporting job reads two Unity Catalog tables, orders and payments, which separate pipelines update at different times each night. The job should run once both tables have been updated, not after either one alone.

How should the trigger be configured?

- A. A table update trigger on both tables with Any table is updated, which holds back the run until the later one of the two pipelines commits
- B. A file arrival trigger on the storage paths of both tables, which starts the job when new data files land under both of those locations
- C. A table update trigger on both tables with Any table is updated and a long wait time after the last change, so the later pipeline finishes first
- D. A table update trigger on both tables with All tables are updated, which fires the run once each monitored table has received a fresh commit

Correct answer: D

D. A table update trigger on both tables with All tables are updated, which fires the run once each monitored table has received a fresh commit

Explanation

Correct answer:

A table update trigger on both tables with All tables are updated, which fires the run once each monitored table has received a fresh commit

Why this is correct:

A table update trigger can watch multiple Unity Catalog tables and require that all monitored tables have updates before it fires. This synchronization condition waits for both independent pipelines' commits instead of starting after the first table changes.

Why the other options are wrong:

A table update trigger on both tables with Any table is updated, which holds back the run until the later one of the two pipelines commits: The job must wait for both independent commits; this setting permits the first table update alone to start a run.

A file arrival trigger on the storage paths of both tables, which starts the job when new data files land under both of those locations: The readiness signal is a table commit, not a file landing at a storage path; the trigger type would monitor the wrong event.

A table update trigger on both tables with Any table is updated and a long wait time after the last change, so the later pipeline finishes first: Adding a delay to an any-table trigger does not guarantee

both tables were updated; the second pipeline may finish after that arbitrary delay.

References:

1. Trigger jobs when source tables are updated:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 9

The Delta table `ops.silver.packages` holds only rows with a positive, non-null `weight_kg`. From now on, any write that includes a zero or negative `weight_kg` must fail instead of being stored.

Which change enforces this rule?

- A. Add a CHECK constraint requiring `weight_kg >= 0`, which rejects the negative values while still accepting the rows where `weight_kg` is zero
- B. Set `weight_kg` to NOT NULL, which in turn makes the table reject any write whose `weight_kg` is missing, zero, or below zero from now on
- C. Delete the rows where `weight_kg` is zero or negative, which removes the invalid values and also stops later writes from adding any more of them
- D. Add a CHECK constraint requiring `weight_kg > 0`, which makes any later transaction that writes even one violating row fail as a whole

Correct answer: D

D. Add a CHECK constraint requiring `weight_kg > 0`, which makes any later transaction that writes even one violating row fail as a whole

Explanation

Correct answer:

Add a CHECK constraint requiring `weight_kg > 0`, which makes any later transaction that writes even one violating row fail as a whole

Why this is correct:

A Delta CHECK constraint is evaluated against rows written to the table, and a transaction that violates it fails instead of committing invalid data. Requiring `weight_kg > 0` rejects zero and negative values while allowing positive non-null weights.

Why the other options are wrong:

Delete the rows where `weight_kg` is zero or negative, which removes the invalid values and also stops later writes from adding any more of them: Deleting existing invalid rows only cleans current data. It does not enforce a rule on later writes, so the same invalid values could be inserted again.

Add a CHECK constraint requiring `weight_kg >= 0`, which rejects the negative values while still accepting the rows where `weight_kg` is zero: `>= 0` allows a weight of zero, which the requirement explicitly says must fail.

Set `weight_kg` to NOT NULL, which in turn makes the table reject any write whose `weight_kg` is missing, zero, or below zero from now on: Nullability is independent of magnitude: this constraint would still allow zero and negative weights.

References:

1. Constraints on Databricks: <https://docs.databricks.com/aws/en/tables/constraints>

Question 10

In a Lakeflow Job, load_eu and load_us run in parallel against regional sources. A downstream merge task must run when either load succeeds, but it must not run when both loads fail.

Which Run if condition should the merge task use?

- A. All succeeded, the default condition, which then starts the merge task only when both regional loads finish the run with a successful final status
- B. All done, which then starts the merge task after both regional loads have finished, whatever final status each of the two loads reports in that job run
- C. None failed, which starts the merge task only when neither regional load reports a failure status and at least one of the two loads has actually been run
- D. At least one succeeded, which then starts the merge task after both loads finish, provided one or both of them completed successfully in that job run

Correct answer: D

D. At least one succeeded, which then starts the merge task after both loads finish, provided one or both of them completed successfully in that job run

Explanation

Correct answer:

At least one succeeded, which then starts the merge task after both loads finish, provided one or both of them completed successfully in that job run

Why this is correct:

The merge task depends on both regional loads, so its Run if condition is evaluated after both finish. “At least one succeeded” permits partial progress from either region while excluding the case where both upstream tasks fail.

Why the other options are wrong:

All done, which then starts the merge task after both regional loads have finished, whatever final status each of the two loads reports in that job run: “All done” checks completion, not success. It would run the merge after two failures and expose the downstream task to missing inputs.

None failed, which starts the merge task only when neither regional load reports a failure status and at least one of the two loads has actually been run: The merge must be allowed to use a successful region even when its peer failed; this condition would block that partial-success case.

All succeeded, the default condition, which then starts the merge task only when both regional loads finish the run with a successful final status: “All succeeded” requires both regions to finish successfully. That is stricter than the stated either-region condition and skips useful work when only one

region fails.

References:

1. Job | Databricks API Reference: <https://docs.databricks.com/api/jobs/v2/job>

Question 11

A finance team works only in SQL and loads a few hundred CSV files a day from one cloud path into a Delta table. Loads must be retry-safe without a streaming checkpoint to manage.

Which ingestion method fits best?

- A.** A daily INSERT INTO from read_files over the whole path, which rereads all of the files and relies on the target to reject the duplicate rows
- B.** Lakeflow Connect with a managed connector for the storage path, which then adds change tracking for each CSV file that the team uploads there
- C.** Auto Loader in file notification mode, which is built to scale up to millions of files per hour while tracking its progress in a streaming checkpoint
- D.** COPY INTO from the cloud path, a SQL command that tracks every file that it has ingested in the table's metadata and then skips them on later runs

Correct answer: D

D. COPY INTO from the cloud path, a SQL command that tracks every file that it has ingested in the table's metadata and then skips them on later runs

Explanation

Correct answer:

COPY INTO from the cloud path, a SQL command that tracks every file that it has ingested in the table's metadata and then skips them on later runs

Why this is correct:

COPY INTO is a SQL file-ingestion command with idempotent tracking in the target Delta table's metadata. Re-running the same statement skips previously loaded files, meeting the retry-safe requirement without a streaming checkpoint.

Why the other options are wrong:

Auto Loader in file notification mode, which is built to scale up to millions of files per hour while tracking its progress in a streaming checkpoint: Auto Loader is optimized for continuously discovering files at scale and maintains streaming progress through a checkpoint. That is more machinery than the SQL-only, modest daily batch requires.

Lakeflow Connect with a managed connector for the storage path, which then adds change tracking for each CSV file that the team uploads there: The connector family is for supported SaaS and database systems, so it will not manage ingestion from an arbitrary object-storage directory.

A daily INSERT INTO from read_files over the whole path, which rereads all of the files and relies on the target to reject the duplicate rows: Without file tracking or a deduplication rule, repeated

INSERT queries append the same source rows again.

References:

1. **COPY INTO:** <https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 12

An engineer committed notebook changes in a Databricks Git folder and pushed them to a remote feature branch. The repository requires a reviewed pull request before anything merges into main.

What should the engineer do next?

- A. Open a pull request from the feature branch to main in the Git provider, where the reviewers can first approve the change before it merges
- B. Merge the feature branch into main within the Git folder and push main, which the Git provider then records as a reviewed code change
- C. Switch the Git folder to main and then pull, which brings the pushed feature commits into main without needing a separate merge step
- D. Open the pull request from the Git folder's branch dialog, which then merges it automatically once the changed notebooks each run without errors

Correct answer: A

A. Open a pull request from the feature branch to main in the Git provider, where the reviewers can first approve the change before it merges

Explanation

Correct answer:

Open a pull request from the feature branch to main in the Git provider, where the reviewers can first approve the change before it merges

Why this is correct:

The feature branch is already pushed, and the repository requires review before changes enter main. Opening a pull request in the Git provider creates the review and approval path; the branch remains separate until an authorized merge.

Why the other options are wrong:

Merge the feature branch into main within the Git folder and push main, which the Git provider then records as a reviewed code change: Merging and pushing directly to main bypasses the repository's required reviewed pull request and approval process.

Switch the Git folder to main and then pull, which brings the pushed feature commits into main without needing a separate merge step: Pull only synchronizes the currently checked-out branch with its upstream. It does not merge feature commits into main.

Open the pull request from the Git folder's branch dialog, which then merges it automatically once the changed notebooks each run without errors: Databricks Git folders synchronize source

code but do not substitute for the Git provider's pull-request review workflow. A successful notebook run also cannot automatically grant approval.

References:

1. CI/CD with Databricks Git folders: <https://docs.databricks.com/aws/en/repos/ci-cd>

Question 13

A bundle declares a variable named `destination_schema`, and each deployment target sets its own value for it. A notebook task in the bundle must receive that value as a parameter.

```
base_parameters:  
  output_schema: _____
```

Which reference completes the parameter?

- A. `${targets.destination_schema}`
- B. `{{job.parameters.destination_schema}}`
- C. `${bundle.destination_schema}`
- D. `${var.destination_schema}`

Correct answer: D

D. `${var.destination_schema}`

Explanation

Correct answer:

`${var.destination_schema}`

Why this is correct:

Bundle variables are deployment-time values, and custom variables are referenced with `${var.<name>}`. Since each target supplies its own `destination_schema` value, this syntax resolves the environment-specific value when the bundle is deployed.

Why the other options are wrong:

`${bundle.destination_schema}`: `${bundle...}` addresses built-in bundle substitutions such as the bundle name or target, not a custom variable declared in the bundle's variables mapping.

`{{job.parameters.destination_schema}}`: `{{job.parameters...}}` is a job-run dynamic reference, not a deployment-time bundle variable. The stem says the value is supplied per deployment target.

`${targets.destination_schema}`: `targets` is a bundle configuration section, not a supported substitution namespace for retrieving a custom variable's resolved value.

References:

1. Substitutions and variables in Declarative Automation Bundles:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 14

An engineer holds **USE CATALOG**, **USE SCHEMA**, and **CREATE TABLE** on a schema and can create managed tables there. Creating a table whose **LOCATION** sits inside an existing external location fails with a permission error.

Which additional grant is needed?

- A. **CREATE EXTERNAL LOCATION** on the metastore, because a table can only use a path that the engineer has defined as an external location
- B. **CREATE EXTERNAL TABLE** on the external location, which permits registering tables whose data files sit under that location's configured storage path
- C. **CREATE MANAGED STORAGE** on the external location, which lets the new tables store their own data files under that location's storage path
- D. **READ FILES** on the external location, which lets the engineer register tables over the data files that are stored under its storage path

Correct answer: B

B. CREATE EXTERNAL TABLE on the external location, which permits registering tables whose data files sit under that location's configured storage path

Explanation

Correct answer:

CREATE EXTERNAL TABLE on the external location, which permits registering tables whose data files sit under that location's configured storage path

Why this is correct:

Creating a table whose **LOCATION** is under an external location requires permission to create an external table on that storage object, in addition to schema-level **CREATE TABLE**. The engineer already has the schema privileges; the missing permission is on the external location.

Why the other options are wrong:

READ FILES on the external location, which lets the engineer register tables over the data files that are stored under its storage path: **READ FILES** permits file access and inspection, but does not authorize registering an external table at that location. Table registration uses **CREATE EXTERNAL TABLE**.

CREATE EXTERNAL LOCATION on the metastore, because a table can only use a path that the engineer has defined as an external location: **CREATE EXTERNAL LOCATION** is used to define a new Unity Catalog external location, not to register a table under one that already exists.

CREATE MANAGED STORAGE on the external location, which lets the new tables store their own

data files under that location's storage path: Managed-storage registration is a different operation; for an explicit table LOCATION, the needed external-location privilege is CREATE EXTERNAL TABLE.

References:

1. Manage external locations:

<https://docs.databricks.com/aws/en/connect/unity-catalog/cloud-storage/manage-external-locations>

Question 15

A notebook calls `collect()` on a large `DataFrame` and fails with a driver out-of-memory error. The engineer needs only 50 sample rows to inspect and cannot change the driver or cluster size.

Which change directly fixes this?

- A. Raise `spark.driver.maxResultSize` so `collect()` may return more of the data, keeping the same driver instance type and cluster size
- B. Apply `limit(50)` before calling `collect()`, because then only the rows that are needed for inspection reach the driver
- C. Call `toPandas()` instead of `collect()`, which converts the rows with Apache Arrow in batches so the driver never holds them all
- D. Call `sample(fraction=0.01)` before `collect()`, which returns about 50 of the rows however large the `DataFrame` happens to be

Correct answer: B

B. Apply `limit(50)` before calling `collect()`, because then only the rows that are needed for inspection reach the driver

Explanation

Correct answer:

Apply `limit(50)` before calling `collect()`, because then only the rows that are needed for inspection reach the driver

Why this is correct:

`limit(50)` bounds the `DataFrame` before collection, so at most the requested sample reaches the driver. This addresses the memory failure at its cause while preserving the requested inspection and leaving the cluster size unchanged.

Why the other options are wrong:

Raise `spark.driver.maxResultSize` so `collect()` may return more of the data, keeping the same driver instance type and cluster size: Raising `spark.driver.maxResultSize` changes a result-size guard, not the driver's available memory. Collecting a large result can still exhaust the driver heap.

Call `toPandas()` instead of `collect()`, which converts the rows with Apache Arrow in batches so the driver never holds them all: `toPandas()` also transfers results to driver memory and materializes them there; changing the conversion method does not make an unbounded result safe.

Call `sample(fraction=0.01)` before `collect()`, which returns about 50 of the rows however large the `DataFrame` happens to be: A 1% sample has no fixed upper bound of 50 rows, so a large input can still send far too much data to the driver.

References:

1. **pyspark.sql.DataFrame.limit**: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.limit.html>

Question 16

A job must run a .sql file of several statements, unchanged from the Git repository, to refresh reporting tables. The job's other tasks use a classic jobs cluster, and a serverless SQL warehouse is available.

How should the SQL file run?

- A. As a notebook task on the jobs cluster, pasting each statement into its own notebook cell so that the job cluster executes the SQL
- B. As a SQL task of the type File that runs on the job's classic jobs cluster, reusing the same compute that the other tasks already share
- C. As a SQL task of the type File that runs on the serverless SQL warehouse, which executes the statements found in the file one after another
- D. As a pipeline task that points at the .sql file itself, which treats each refresh statement in it as part of one declarative pipeline definition

Correct answer: C

C. As a SQL task of the type File that runs on the serverless SQL warehouse, which executes the statements found in the file one after another

Explanation

Correct answer:

As a SQL task of the type File that runs on the serverless SQL warehouse, which executes the statements found in the file one after another

Why this is correct:

A SQL task configured as a File runs a repository .sql file against a Databricks SQL warehouse and supports multiple semicolon-separated statements. A serverless warehouse is available, so the job can execute the file without converting it to notebook code.

Why the other options are wrong:

As a SQL task of the type File that runs on the job's classic jobs cluster, reusing the same compute that the other tasks already share: A notebook task would require wrapping or copying the SQL into a notebook, changing the asset form the team wants to preserve.

As a notebook task on the jobs cluster, pasting each statement into its own notebook cell so that the job cluster executes the SQL: A Python wheel task runs a packaged Python entry point. It does not interpret a SQL file as a sequence of warehouse statements.

As a pipeline task that points at the .sql file itself, which treats each refresh statement in it as part of one declarative pipeline definition: A Run Job task starts a different Lakeflow Job; it is not a task for

directly executing a SQL source file.

References:

1. **SQL task for jobs:** <https://docs.databricks.com/aws/en/jobs/tasks/sql>

Question 17

The file `events.json` in the volume `lab.landing.inbox` holds one JSON array spread over many lines rather than one object per line. An engineer wants to query it directly with SQL, without creating a table first.

Which statement returns one row per array element?

- A. `SELECT * FROM lab.landing.inbox.events;`
- B. `SELECT * FROM read_files('/Volumes/lab/landing/inbox/events.json', format => 'text');`
- C. `SELECT * FROM read_files('/Volumes/lab/landing/inbox/events.json', format => 'json', multiLine => true);`
- D. `SELECT * FROM json./Volumes/lab/landing/inbox/events.json;`

Correct answer: C

C. `SELECT * FROM read_files('/Volumes/lab/landing/inbox/events.json', format => 'json', multiLine => true);`

Explanation

Correct answer:

`SELECT * FROM read_files('/Volumes/lab/landing/inbox/events.json', format => 'json', multiLine => true);`

Why this is correct:

`read_files` can query a single JSON file directly from a Unity Catalog volume, and `multiLine => true` tells the JSON reader that one record may span lines. This avoids creating a table first while parsing the file as one JSON document.

Why the other options are wrong:

SELECT * FROM json./Volumes/lab/landing/inbox/events.json;; A plain `read_files` call without multiline mode assumes newline-delimited JSON records, which does not match the file's one array spread over multiple lines.

SELECT * FROM read_files('/Volumes/lab/landing/inbox/events.json', format => 'text'); `binaryFile` returns file metadata and raw content rather than parsing the JSON array into relational columns.

SELECT * FROM lab.landing.inbox.events;; `COPY INTO` loads file data into a target table. The question asks to query the file directly without creating a table.

References:

1. Read and write JSON files: <https://docs.databricks.com/aws/en/query/formats/json>

Question 18

A row filter on `sales.orders` returns `TRUE` when a row's region matches a group of the current user. A user only in the `emea` group, with `SELECT` on the table, runs `SELECT * FROM sales.orders`.

What does the user get?

- A. Every row of the table, with region masked on the rows outside `emea`, because row filters hide values within rows rather than whole rows
- B. An error stating that some rows are restricted, because a row filter blocks any query that would read the rows that the user cannot see
- C. Every row of the table, because the `SELECT` privilege on the table takes precedence over any row filter that is defined on that table
- D. The `emea`-region orders alone, because the non-matching rows are just silently omitted from the output and the query raises no error

Correct answer: D

D. The `emea`-region orders alone, because the non-matching rows are just silently omitted from the output and the query raises no error

Explanation

Correct answer:

The `emea`-region orders alone, because the non-matching rows are just silently omitted from the output and the query raises no error

Why this is correct:

A Unity Catalog row filter evaluates its predicate for each row and returns only rows for which the predicate is true. The `emea`-only user's nonmatching rows are silently omitted; `SELECT` remains allowed on the visible subset.

Why the other options are wrong:

An error stating that some rows are restricted, because a row filter blocks any query that would read the rows that the user cannot see: A row filter is designed to suppress rows from results, not fail the whole query because inaccessible rows exist in the table.

Every row of the table, with region masked on the rows outside `emea`, because row filters hide values within rows rather than whole rows: Masking changes column values while retaining rows. The configured control is a row filter, so nonmatching records are excluded altogether.

Every row of the table, because the `SELECT` privilege on the table takes precedence over any row filter that is defined on that table: `SELECT` grants permission to issue reads, subject to row-level policy

evaluation; it does not expose rows the filter denies.

References:

1. Row filters and column masks:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 19

An Auto Loader stream must keep its current schema when new JSON fields appear, keep running without stopping, and preserve the unexpected fields for later inspection.

```
(spark.readStream.format("cloudFiles")  
  .option("cloudFiles.format", "json")  
  .option("cloudFiles.schemaEvolutionMode", "_____")  
  .load(path))
```

Which mode belongs in the blank?

- A. rescue, which never evolves the schema and instead writes the unknown fields into the rescued data column as the records keep flowing
- B. addNewColumns, which first adds the new fields to the schema and then stops the stream once, after which it restarts with the wider schema
- C. none, which ignores the new fields entirely, keeping the existing schema fixed and the stream running without stopping for them
- D. failOnNewColumns, which stops the stream as soon as the new fields appear and keeps it stopped until someone updates the schema by hand

Correct answer: A

A. rescue, which never evolves the schema and instead writes the unknown fields into the rescued data column as the records keep flowing

Explanation

Correct answer:

rescue, which never evolves the schema and instead writes the unknown fields into the rescued data column as the records keep flowing

Why this is correct:

Auto Loader's rescue schema-evolution mode leaves the declared/inferred schema unchanged, keeps the stream running as new fields arrive, and places unrecognized fields in the rescued data column. Those properties meet all three requirements together.

Why the other options are wrong:

addNewColumns, which first adds the new fields to the schema and then stops the stream once, after which it restarts with the wider schema: addNewColumns updates the stored schema when a new column appears, then stops with an unknown-field exception so the stream can restart with the expanded schema.

failOnNewColumns, which stops the stream as soon as the new fields appear and keeps it

stopped until someone updates the schema by hand: failOnNewColumns deliberately stops on an unexpected field and does not update the schema automatically, contradicting the requirement to keep processing.

none, which ignores the new fields entirely, keeping the existing schema fixed and the stream running without stopping for them: Although the stream continues with a fixed schema, this mode discards new fields unless rescue is separately configured, losing the data the stem asks to retain.

References:

1. Auto Loader best practices:

<https://docs.databricks.com/gcp/en/ingestion/cloud-object-storage/auto-loader/best-practices>

Question 20

An engineer cleans a batch DataFrame with the following line, passing no other arguments:

```
clean_df = raw_df.dropna(subset=["shipment_id", "destination"])
```

Which rows does this remove?

- A. Rows in which both shipment_id and destination are null, while any row that still has a non-null value in either of the two columns is kept
- B. Rows in which shipment_id, destination, or both are null, because the default how setting drops a row when any of the listed columns holds a null
- C. Rows in which any column of the DataFrame holds a null, because the subset argument only sets the order in which the listed columns are checked
- D. Rows in which shipment_id or destination holds an empty string, since dropna treats blank text in those two columns the same way as it treats missing values

Correct answer: B

B. Rows in which shipment_id, destination, or both are null, because the default how setting drops a row when any of the listed columns holds a null

Explanation

Correct answer:

Rows in which shipment_id, destination, or both are null, because the default how setting drops a row when any of the listed columns holds a null

Why this is correct:

subset limits dropna's null check to shipment_id and destination. Since the default is how="any", a row is dropped when either listed column is null; a null in another column does not matter.

Why the other options are wrong:

Rows in which both shipment_id and destination are null, while any row that still has a non-null value in either of the two columns is kept: Dropping only when both checked fields are null requires how="all". The default how="any" removes rows as soon as one selected field is null.

Rows in which shipment_id or destination holds an empty string, since dropna treats blank text in those two columns the same way as it treats missing values: An empty string is a present string value, not a null. dropna will keep it unless the code separately filters blank text.

Rows in which any column of the DataFrame holds a null, because the subset argument only sets the order in which the listed columns are checked: Columns outside shipment_id and destination are ignored, and the method does not interpret the subset as a checking order.

References:

1. **pyspark.sql.DataFrameNaFunctions.drop**: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrameNaFunctions.drop.html>

Question 21

The data_stewards group must grant and revoke SELECT on sales.gold.revenue for other groups. Stewards already hold USE CATALOG and USE SCHEMA, ownership stays with platform_admins, and stewards need no data privileges on the table.

What should an admin grant to data_stewards?

- A. ALL PRIVILEGES on the table, which includes both SELECT and MODIFY and lets the stewards pass those same privileges on to any other group that needs them
- B. MANAGE on the table, which lets the stewards grant and revoke privileges on it without ownership of it being transferred to their group
- C. Ownership of the table, transferred to them with ALTER TABLE, so stewards can grant privileges on it while platform_admins keep all of their current grants
- D. SELECT on the table WITH GRANT OPTION, which lets the stewards read the table data themselves and delegate that same privilege to other groups

Correct answer: B

B. MANAGE on the table, which lets the stewards grant and revoke privileges on it without ownership of it being transferred to their group

Explanation

Correct answer:

MANAGE on the table, which lets the stewards grant and revoke privileges on it without ownership of it being transferred to their group

Why this is correct:

MANAGE lets a principal grant and revoke privileges on the table without transferring ownership. The stewards already have the parent USE CATALOG and USE SCHEMA privileges, and MANAGE does not itself give them table data access.

Why the other options are wrong:

Ownership of the table, transferred to them with ALTER TABLE, so stewards can grant privileges on it while platform_admins keep all of their current grants: Ownership would grant broad control over the table and contradicts the explicit requirement that platform_admins retain ownership. The narrower administrative privilege is sufficient.

ALL PRIVILEGES on the table, which includes both SELECT and MODIFY and lets the stewards pass those same privileges on to any other group that needs them: ALL PRIVILEGES does not include MANAGE, so it does not give the stewards authority to delegate access. It would also grant data capabilities they do not need.

SELECT on the table WITH GRANT OPTION, which lets the stewards read the table data themselves and delegate that same privilege to other groups: Delegating SELECT through a grant option would also give the stewards data access, and it is narrower than the requested grant-and-revoke administration.

References:

1. Manage privileges in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/>

Question 22

A managed Delta table is filtered on several high-cardinality columns that shift as query patterns evolve. The team wants data skipping, no fixed partition hierarchy, and no full rewrite of existing data when the layout columns change.

Which layout approach fits best?

- A. Run OPTIMIZE ZORDER BY on the current filter columns, because a later OPTIMIZE can then switch to new columns whenever the patterns shift
- B. Enable liquid clustering on the current filter columns, because ALTER TABLE CLUSTER BY can switch the keys, clustering only the new or re-optimized data
- C. Partition by date and Z-order each partition on the other filter columns, because the Z-order columns can differ on each new OPTIMIZE run
- D. Partition the table by every column used in query filters, so each query then reads only the directories that match its filter values

Correct answer: B

B. Enable liquid clustering on the current filter columns, because ALTER TABLE CLUSTER BY can switch the keys, clustering only the new or re-optimized data

Explanation

Correct answer:

Enable liquid clustering on the current filter columns, because ALTER TABLE CLUSTER BY can switch the keys, clustering only the new or re-optimized data

Why this is correct:

Liquid clustering organizes data for skipping without a fixed partition hierarchy and supports changing the clustering keys with ALTER TABLE. Changing keys does not immediately rewrite existing files; subsequent optimization and writes apply the new layout incrementally, fitting shifting filter patterns.

Why the other options are wrong:

Partition the table by every column used in query filters, so each query then reads only the directories that match its filter values: Partitioning on several high-cardinality filters creates a rigid directory structure with many small partitions and is difficult to adapt as query patterns change.

Partition by date and Z-order each partition on the other filter columns, because the Z-order columns can differ on each new OPTIMIZE run: Changing the Z-order column list requires explicit optimization work to rewrite layout; the requested keys should be easier to change as patterns evolve.

Run OPTIMIZE ZORDER BY on the current filter columns, because a later OPTIMIZE can then switch to new columns whenever the patterns shift: This layout does not offer the same flexible

clustering-key configuration; adapting it requires another explicit Z-order optimization over files.

References:

1. Use liquid clustering for tables: <https://docs.databricks.com/aws/en/delta/clustering>

Question 23

An Auto Loader job must process all files present when each run starts, using as many micro-batches as needed, and then stop. Files that arrive later wait for the next run.

```
query = (incoming.writeStream
  .option("checkpointLocation", checkpoint_path)
  ._____
  .toTable("ops.bronze.events"))
```

Which trigger fits?

- A. `trigger(processingTime="0 seconds")`, which processes the backlog as fast as possible and then stops once no unprocessed files remain
- B. `trigger(continuous="1 second")`, which reads the backlog with low latency and then shuts down once every file has been processed
- C. `trigger(once=True)`, which processes the files present at start in a single micro-batch, a setting now deprecated in favor of newer options
- D. `trigger(availableNow=True)`, which processes the whole existing backlog, splitting it up as needed, and afterward ends the query

Correct answer: D

D. `trigger(availableNow=True)`, which processes the whole existing backlog, splitting it up as needed, and afterward ends the query

Explanation

Correct answer:

`trigger(availableNow=True)`, which processes the whole existing backlog, splitting it up as needed, and afterward ends the query

Why this is correct:

`availableNow=True` processes the data available to the query in one or more micro-batches, then terminates. Files arriving after the run begins are not part of that available-now backlog and can be processed by the next run.

Why the other options are wrong:

`trigger(once=True)`, which processes the files present at start in a single micro-batch, a setting now deprecated in favor of newer options: `once=True` processes available input in a single micro-batch and is deprecated in favor of `AvailableNow`; it does not split a large backlog across as many batches as needed.

`trigger(processingTime="0 seconds")`, which processes the backlog as fast as possible and then

stops once no unprocessed files remain: A zero-second processing-time trigger keeps scheduling micro-batches while the stream is active. It does not itself define the finite available-now run requested.

trigger(continuous="1 second"), which reads the backlog with low latency and then shuts down once every file has been processed: Continuous processing is a low-latency streaming mode, not a bounded backlog mode that automatically stops after consuming current files.

References:

1. Configure Structured Streaming trigger intervals:

<https://docs.databricks.com/gcp/en/structured-streaming/triggers>

Question 24

Predictive optimization is enabled for a catalog's Unity Catalog managed tables, and the team wants to stop scheduling routine maintenance jobs for those tables.

Which TWO operations can predictive optimization run automatically?

- A. Running ANALYZE to collect the table statistics, which the query optimizer then uses when it plans the queries
- B. Running OPTIMIZE with ZORDER BY on the columns that queries against the tables filter on most often
- C. Running OPTIMIZE to compact the small data files, timed according to each table's own write and query activity
- D. Running REORG TABLE to rewrite the data files and purge the rows that deletion vectors soft-deleted
- E. Running OPTIMIZE on external tables that are registered in the same catalog as the managed tables

Correct answers: A, C

- A. Running ANALYZE to collect the table statistics, which the query optimizer then uses when it plans the queries
- C. Running OPTIMIZE to compact the small data files, timed according to each table's own write and query activity

Explanation

Correct answer:

Running OPTIMIZE to compact the small data files, timed according to each table's own write and query activity

Running ANALYZE to collect the table statistics, which the query optimizer then uses when it plans the queries

Why this is correct:

Predictive optimization can run OPTIMIZE to compact and lay out files and ANALYZE to refresh optimizer statistics for Unity Catalog managed tables. Those automated maintenance operations cover both file organization and query-planning information.

Why the other options are wrong:

Running OPTIMIZE with ZORDER BY on the columns that queries against the tables filter on most often: Predictive optimization runs OPTIMIZE, but it does not choose or run ZORDER BY on selected columns as part of that automatic operation. Liquid clustering is the managed layout feature for evolving filter patterns.

Running OPTIMIZE on external tables that are registered in the same catalog as the managed tables: The feature targets Unity Catalog managed tables; external tables registered in the same catalog are outside its automatic maintenance scope.

Running REORG TABLE to rewrite the data files and purge the rows that deletion vectors soft-deleted: Deletion-vector purging is a separate explicit rewrite, outside the routine predictive-optimization operations at issue here.

References:

1. Predictive optimization for Unity Catalog managed tables:

<https://docs.databricks.com/gcp/en/optimizations/predictive-optimization>

Question 25

An engineer configures Auto Loader to infer the schema of JSON files, but the stream fails to start because an option that inference and evolution depend on is missing.

```
incoming = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("_____", schema_path)
  .load(source_path))
```

Which option belongs in the blank?

- A. checkpointLocation
- B. cloudFiles.schemaLocation
- C. cloudFiles.schemaHints
- D. cloudFiles.inferColumnTypes

Correct answer: B

B. cloudFiles.schemaLocation

Explanation

Correct answer:

cloudFiles.schemaLocation

Why this is correct:

cloudFiles.schemaLocation stores the inferred schema and its evolution history so Auto Loader can reuse it across stream restarts. Inference and evolution depend on this persisted schema path so Auto Loader can reuse the same schema across restarts.

Why the other options are wrong:

checkpointLocation: checkpointLocation stores Structured Streaming progress and state for a streaming query. Auto Loader's inferred schema history has its own cloudFiles.schemaLocation option.

cloudFiles.schemaHints: Schema hints specify desired types or structure for columns; they do not identify where Auto Loader persists its inferred and evolving schema.

cloudFiles.inferColumnTypes: Column type inference settings affect how values are interpreted for formats such as CSV and JSON; they do not identify where Auto Loader persists schema history.

References:

1. Spark API options reference: <https://docs.databricks.com/aws/en/spark/api-options>

Question 26

Analysts need a named object in `corp.gold` that always shows current totals from `corp.silver.sales` at query time. The team wants no stored copy of the results and no refresh schedule to manage.

Which object fits?

- A. A materialized view, which stores the precomputed totals and refreshes them on a schedule, so the queries read the stored results
- B. A streaming table over `corp.silver.sales`, which stores the totals and then updates them incrementally whenever its own pipeline runs
- C. A standard view, which saves only its query and then computes the totals from the source data each time it is queried
- D. A temporary view, which computes the totals at query time but exists only within the Spark session that created it

Correct answer: C

C. A standard view, which saves only its query and then computes the totals from the source data each time it is queried

Explanation

Correct answer:

A standard view, which saves only its query and then computes the totals from the source data each time it is queried

Why this is correct:

A standard view stores its query definition rather than a materialized result, so the source is evaluated when the view is queried. That yields current totals without a stored copy or a refresh schedule.

Why the other options are wrong:

A materialized view, which stores the precomputed totals and refreshes them on a schedule, so the queries read the stored results: Precomputed results must be refreshed as source data changes, adding the stored copy and maintenance cycle the team wants to avoid.

A streaming table over `corp.silver.sales`, which stores the totals and then updates them incrementally whenever its own pipeline runs: A streaming table persists data and updates through a pipeline; it is not a virtual query-only object computed afresh at each read.

A temporary view, which computes the totals at query time but exists only within the Spark session that created it: Session-scoped metadata disappears outside its creating session, so it cannot be the persistent object analysts need in `corp.gold`.

References:

1. CREATE VIEW:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-view>

Question 27

A notebook ingests records from a REST API that returns at most 500 records per response and includes a `next_page_token` while more remain. Only the first response is written, so the bronze table is incomplete.

Which change retrieves the full result set?

- A. Fetch the pages in parallel from the Spark executors by handing each of the executors a different `next_page_token` value in advance
- B. Keep requesting while each response holds exactly 500 records, passing the latest `next_page_token` value along with every new request
- C. Increment a numeric page query parameter from 1 upward until a response comes back empty, writing each page to storage as it arrives
- D. Keep requesting pages with the `next_page_token` value from each previous response, writing every page out, until a response carries no token

Correct answer: D

D. Keep requesting pages with the `next_page_token` value from each previous response, writing every page out, until a response carries no token

Explanation

Correct answer:

Keep requesting pages with the `next_page_token` value from each previous response, writing every page out, until a response carries no token

Why this is correct:

The API's `next_page_token` is a continuation cursor. The client must pass each returned token into the next request and keep writing pages until no token is returned; stopping after page one leaves later records unread.

Why the other options are wrong:

Increment a numeric page query parameter from 1 upward until a response comes back empty, writing each page to storage as it arrives: This assumes the API uses numeric page numbers, but the response supplies an opaque continuation token. Advancing a page parameter may repeat or skip data.

Keep requesting while each response holds exactly 500 records, passing the latest `next_page_token` value along with every new request: A final page may contain fewer than 500 records and still include the cursor to another page. The token, not page fullness, determines whether to continue.

Fetch the pages in parallel from the Spark executors by handing each of the executors a different

next_page_token value in advance: The next token depends on the previous response, so it cannot be safely preassigned to independent executors. Sequential cursor retrieval is required unless the API offers separate partition cursors.

References:

1. Query: <https://developer.salesforce.com/docs/platform/api-rest/guide/resources-query.html>

Question 28

An engineer must create a table in ops.raw that Unity Catalog manages, with its data stored in the schema's managed storage location. The engineer has the needed privileges, and the table ops.raw.staging exists.

Which TWO statements create a managed table?

- A. `CREATE TEMPORARY VIEW events AS SELECT event_id FROM ops.raw.staging;`
- B. `CREATE TABLE ops.raw.events (event_id BIGINT) LOCATION 's3://company-data/events/';`
- C. `CREATE TABLE ops.raw.events AS SELECT event_id FROM ops.raw.staging;`
- D. `CREATE VIEW ops.raw.events AS SELECT event_id FROM ops.raw.staging;`
- E. `CREATE TABLE ops.raw.events (event_id BIGINT, event_ts TIMESTAMP, source STRING);`

Correct answers: C, E

- C. `CREATE TABLE ops.raw.events AS SELECT event_id FROM ops.raw.staging;`
- E. `CREATE TABLE ops.raw.events (event_id BIGINT, event_ts TIMESTAMP, source STRING);`

Explanation

Correct answer:

`CREATE TABLE ops.raw.events (event_id BIGINT, event_ts TIMESTAMP, source STRING);`

`CREATE TABLE ops.raw.events AS SELECT event_id FROM ops.raw.staging;`

Why this is correct:

Unity Catalog creates a managed table when `CREATE TABLE` omits an external `LOCATION`, storing its data in the schema's managed storage. Both an explicit schema definition and CTAS without `LOCATION` create managed tables, as long as the object type is a table.

Why the other options are wrong:

`CREATE TABLE ops.raw.events (event_id BIGINT) LOCATION 's3://company-data/events/';` An explicit cloud `LOCATION` makes this an external table: Unity Catalog registers the metadata but the files are stored at the specified path.

`CREATE VIEW ops.raw.events AS SELECT event_id FROM ops.raw.staging;` A view stores a query

definition rather than table data, so it is not a managed table even though it references an existing table.

CREATE TEMPORARY VIEW events AS SELECT event_id FROM ops.raw.staging;; A temporary view is session-scoped query metadata, not a persistent Unity Catalog managed table.

References:

1. Unity Catalog managed tables for Delta Lake and Apache Iceberg:

<https://docs.databricks.com/aws/en/tables/managed>

Question 29

A team sets up a Lakeflow Connect managed connector for an on-premises SQL Server database with change tracking enabled. Changes must flow continuously into Unity Catalog Delta tables.

Which TWO components does this connector setup create?

- A. A foreign catalog through Lakehouse Federation, which queries the source SQL Server tables in place on every read
- B. A serverless SQL warehouse that runs scheduled MERGE statements from the changed rows into the Delta tables
- C. An ingestion pipeline on serverless compute, which then applies the captured changes onto the destination Delta tables
- D. A Structured Streaming job on classic compute that polls the change tables and then stages the rows in a volume
- E. An ingestion gateway on classic compute, which runs continuously to capture the source changes before the logs are truncated

Correct answers: C, E

C. An ingestion pipeline on serverless compute, which then applies the captured changes onto the destination Delta tables

E. An ingestion gateway on classic compute, which runs continuously to capture the source changes before the logs are truncated

Explanation

Correct answer:

An ingestion gateway on classic compute, which runs continuously to capture the source changes before the logs are truncated

An ingestion pipeline on serverless compute, which then applies the captured changes onto the destination Delta tables

Why this is correct:

The SQL Server managed connector separates change capture from destination application: a gateway continuously extracts source changes on classic compute, while a Lakeflow ingestion pipeline on serverless compute applies staged changes to destination Delta tables. Change tracking makes this capture path incremental.

Why the other options are wrong:

A Structured Streaming job on classic compute that polls the change tables and then stages the rows in a volume: The managed SQL Server connector uses an ingestion gateway and pipeline, not a

user-managed Structured Streaming poller that stages rows in a volume.

A serverless SQL warehouse that runs scheduled MERGE statements from the changed rows into the Delta tables: The connector does not create a scheduled SQL warehouse task; its managed architecture uses a gateway and ingestion pipeline.

A foreign catalog through Lakehouse Federation, which queries the source SQL Server tables in place on every read: A foreign catalog provides query-in-place access and does not continuously land source changes into destination Delta tables.

References:

1. Ingest data from SQL Server:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/sql-server-pipeline>

Question 30

Two SQL queries return the same columns in the same order. Their results must be combined into one output that keeps every row, including rows duplicated within or across the two queries.

Which operator should connect the queries?

- A. `unionByName` on the two DataFrames, which matches the columns by name and removes the duplicate rows from the combined output
- B. `UNION ALL`, which stacks the two results as they are and keeps each row, including the duplicates, without comparing the rows
- C. `UNION DISTINCT`, which stacks both of the results, keeping the rows that repeat within one query and collapsing repeats across the two
- D. `UNION`, which stacks the two results and then removes the duplicate rows so each distinct row appears just once in the combined output

Correct answer: B

B. `UNION ALL`, which stacks the two results as they are and keeps each row, including the duplicates, without comparing the rows

Explanation

Correct answer:

`UNION ALL`, which stacks the two results as they are and keeps each row, including the duplicates, without comparing the rows

Why this is correct:

`UNION ALL` appends the rows from both query results without eliminating duplicates. That preserves repeated rows within either input as well as rows that appear in both, which is the explicit result contract.

Why the other options are wrong:

`UNION`, which stacks the two results and then removes the duplicate rows so each distinct row appears just once in the combined output: Plain `UNION` is distinct by default, so it deduplicates the combined result and loses repetitions the question requires.

`UNION DISTINCT`, which stacks both of the results, keeping the rows that repeat within one query and collapsing repeats across the two: `UNION DISTINCT` explicitly requests duplicate elimination. It does not preserve duplicates within one input or between the inputs.

`unionByName` on the two DataFrames, which matches the columns by name and removes the duplicate rows from the combined output: `unionByName` is a DataFrame API for aligning columns by name, but it performs distinct union semantics unless the corresponding `all` operation is used.

References:

1. Set operators:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-select-setops>

Question 31

A batch aggregation spills to disk on every run. The Spark UI shows all 20 shuffle partitions at similar, large sizes, while idle cores could run far more tasks at once.

Which change should the engineer test first, then remeasure?

- A. Raise `spark.sql.shuffle.partitions` well above 20 so each task handles less of the data, then recheck the spill and duration for that stage
- B. Add a few more worker nodes so the 20 shuffle tasks have more total memory to share, then recheck the spill and duration for that stage
- C. Lower `spark.sql.shuffle.partitions` to 10 so each task takes on a larger share of the data, then recheck the spill and duration for that stage
- D. Coalesce the input DataFrame to fewer partitions before the aggregation runs, giving each task a larger and more even share of the input rows

Correct answer: A

A. Raise `spark.sql.shuffle.partitions` well above 20 so each task handles less of the data, then recheck the spill and duration for that stage

Explanation

Correct answer:

Raise `spark.sql.shuffle.partitions` well above 20 so each task handles less of the data, then recheck the spill and duration for that stage

Why this is correct:

The 20 shuffle partitions are similarly large, and available cores are idle, so increasing `spark.sql.shuffle.partitions` is the direct first experiment. More, smaller tasks can use the parallel capacity and reduce per-task memory pressure; remeasurement determines whether it helps this workload.

Why the other options are wrong:

Lower `spark.sql.shuffle.partitions` to 10 so each task takes on a larger share of the data, then recheck the spill and duration for that stage: Reducing the partition count makes each shuffle task handle more data, increasing the memory pressure behind the existing spills and leaving even more cores idle.

Coalesce the input DataFrame to fewer partitions before the aggregation runs, giving each task a larger and more even share of the input rows: Coalescing before the aggregation reduces input parallelism and creates larger upstream partitions. The observed issue is large shuffle tasks, for which increasing shuffle partitions is the targeted adjustment.

Add a few more worker nodes so the 20 shuffle tasks have more total memory to share, then

recheck the spill and duration for that stage: More workers add capacity, but the stem already identifies idle cores. The partitions themselves are too large, so changing their count is the more direct test before scaling the cluster.

References:

1. Adaptive query execution: <https://docs.databricks.com/aws/en/optimizations/adaptive-query-execution>

Question 32

A daily COPY INTO with default options loaded 20 Parquet files yesterday. Overnight, the vendor replaced one of those files with a corrected version under the same name and added five new files. The command runs again.

What does this run load?

- A. All 25 files, because a changed file in the source path makes COPY INTO treat the directory as new and then reload every file that it contains
- B. Nothing at all, because a modified file that was loaded in a prior run makes the command stop with a conflict until the table is restored
- C. The five new files plus the corrected file, because COPY INTO then reloads any previously loaded file whose file content has changed since then
- D. Only the five new files, because files that were already loaded are skipped even after they change, unless the force copy option is set to true

Correct answer: D

D. Only the five new files, because files that were already loaded are skipped even after they change, unless the force copy option is set to true

Explanation

Correct answer:

Only the five new files, because files that were already loaded are skipped even after they change, unless the force copy option is set to true

Why this is correct:

COPY INTO tracks files already loaded into the Delta target and skips them on later runs, even if a source file was modified. With default options, the corrected file is still considered seen, while the five new filenames are ingested.

Why the other options are wrong:

The five new files plus the corrected file, because COPY INTO then reloads any previously loaded file whose file content has changed since then: COPY INTO does not compare file contents to detect corrections after a successful load; reprocessing previously seen files requires setting the force option to true.

All 25 files, because a changed file in the source path makes COPY INTO treat the directory as new and then reload every file that it contains: A directory change does not reset COPY INTO's file history; already loaded files remain skipped while unseen files are processed.

Nothing at all, because a modified file that was loaded in a prior run makes the command stop

with a conflict until the table is restored: A changed file is not a conflict that blocks the command. The normal behavior is to skip it and continue with new files.

References:

1. Common data loading patterns using COPY INTO:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/copy-into/examples>

Question 33

A CI pipeline builds the team's ETL code into a package exposing a console entry point named `run_etl`. A Lakeflow Job must call it with arguments, without adding any notebook or script file.

Which task type should the job use?

- A. A Python script task whose script file imports `run_etl` from the installed package and then passes the job's arguments through to it
- B. A notebook task that installs the wheel as a cluster library, imports the package, and then calls `run_etl` from the first notebook cell
- C. A Python script task that points at the wheel file itself, which then lets the job run any entry point that the wheel's own metadata defines
- D. A Python wheel task that names the package and the `run_etl` entry point and then supplies the arguments, as positional or as named parameters

Correct answer: D

D. A Python wheel task that names the package and the `run_etl` entry point and then supplies the arguments, as positional or as named parameters

Explanation

Correct answer:

A Python wheel task that names the package and the `run_etl` entry point and then supplies the arguments, as positional or as named parameters

Why this is correct:

A Python wheel task directly invokes a packaged Python entry point, and it accepts positional or keyword arguments. The wheel already exposes `run_etl`, so this task type calls the package without adding a notebook or wrapper script.

Why the other options are wrong:

A notebook task that installs the wheel as a cluster library, imports the package, and then calls `run_etl` from the first notebook cell: A notebook wrapper would add the notebook the requirement explicitly avoids and would require custom code to import and invoke the package.

A Python script task that points at the wheel file itself, which then lets the job run any entry point that the wheel's own metadata defines: The job would look for a Python source file to launch; it cannot resolve the wheel metadata entry point as its executable asset.

A Python script task whose script file imports `run_etl` from the installed package and then passes the job's arguments through to it: A JAR task launches JVM code. The program is a Python package, so the corresponding task should use Python's wheel and entry-point support.

References:

1. Python wheel task for jobs: <https://docs.databricks.com/aws/en/jobs/tasks/python-wheel>

Question 34

A job writing to a Delta table fails partway through, after some data files were written but before its commit reached the transaction log. A report query then reads the table.

What does the report see, and why?

- A. Only the last committed version, because readers locate the data files through the transaction log and the failed write itself never committed
- B. The committed rows plus the rows from each data file that the failed job finished, because Delta commits every completed file on its own
- C. The last committed version, but only after the failed job's retry ends, because a pending log entry blocks the readers until that point
- D. The rows in the files the failed job already wrote, because Delta exposes new data files as soon as they first land in the table's storage path

Correct answer: A

A. Only the last committed version, because readers locate the data files through the transaction log and the failed write itself never committed

Explanation

Correct answer:

Only the last committed version, because readers locate the data files through the transaction log and the failed write itself never committed

Why this is correct:

Delta readers use the transaction log to select a committed table version, rather than treating every file in the storage directory as visible data. Because the failed write did not commit, a query sees the last successful snapshot and ignores its uncommitted files.

Why the other options are wrong:

The rows in the files the failed job already wrote, because Delta exposes new data files as soon as they first land in the table's storage path: Writing a data file alone does not add it to the table snapshot. Without a transaction-log commit, that file is not part of the version readers resolve.

The last committed version, but only after the failed job's retry ends, because a pending log entry blocks the readers until that point: Readers are not blocked waiting for a retry; they continue querying the last committed snapshot while an unsuccessful write remains uncommitted.

The committed rows plus the rows from each data file that the failed job finished, because Delta commits every completed file on its own: Delta commits a transaction atomically through its log; it does not make each completed output file independently visible as a separate table commit.

References:

1. What is Delta Lake in Databricks?: <https://docs.databricks.com/aws/en/delta>

Question 35

A task named `list_stores` sets a task value named `stores`, a JSON array of store IDs that changes every run. A For each task must run a notebook once per store in that array.

What should the For each task's Inputs field contain?

- A. `{{job.parameters.stores}}`, which reads the store list from a job parameter that the `list_stores` task fills in while it runs
- B. `{{tasks.stores.values.list_stores}}`, which names the task value first and then the task that produced it earlier in the run
- C. A JSON array literal copied from yesterday's `list_stores` output, which keeps the For each input fixed in place until someone edits it again
- D. `{{tasks.list_stores.values.stores}}`, which resolves at run time to the array that `list_stores` set earlier within this run

Correct answer: D

D. `{{tasks.list_stores.values.stores}}`, which resolves at run time to the array that `list_stores` set earlier within this run

Explanation

Correct answer:

`{{tasks.list_stores.values.stores}}`, which resolves at run time to the array that `list_stores` set earlier within this run

Why this is correct:

Task values are referenced with the upstream task key followed by `.values.` and the value key. The For each input resolves that dynamic reference at run time to the JSON array created by `list_stores`, so one iteration can run per store.

Why the other options are wrong:

`{{tasks.stores.values.list_stores}}`, which names the task value first and then the task that produced it earlier in the run: The namespace order is reversed: task values use `{{tasks.<task_name>.values.<value_name>}}`, with the task name before its value key.

`{{job.parameters.stores}}`, which reads the store list from a job parameter that the `list_stores` task fills in while it runs: Job parameters are supplied to the run; the upstream notebook instead publishes a task value. The job parameter reference will not read the value set by `list_stores`.

A JSON array literal copied from yesterday's `list_stores` output, which keeps the For each input fixed in place until someone edits it again: A copied literal would become stale when the list changes. The task value reference carries this run's actual generated array.

References:

1. Dynamic value references: <https://docs.databricks.com/aws/en/jobs/dynamic-value-references>

Question 36

Every column tagged pii, in current and future tables across a catalog, must return a masked value to non-privileged readers. Governance wants to define the rule once instead of relying on table owners.

Which approach meets this requirement?

- A. A nightly job that finds the newly tagged columns and runs ALTER COLUMN SET MASK on each one, so the new tables are covered within a day
- B. A governed tag policy on pii that limits who is allowed to assign the tag, so that only the governance team decides which columns are masked
- C. A column mask policy on the catalog that matches the pii tag through ABAC and then masks the column's value wherever that tag is applied
- D. A row filter policy on the catalog that matches the pii tag, which hides the rows containing the tagged values from non-privileged readers

Correct answer: C

C. A column mask policy on the catalog that matches the pii tag through ABAC and then masks the column's value wherever that tag is applied

Explanation

Correct answer:

A column mask policy on the catalog that matches the pii tag through ABAC and then masks the column's value wherever that tag is applied

Why this is correct:

A catalog-scoped ABAC column-mask policy can match columns by governed tag and apply the mask centrally. Because the policy is evaluated against tagged data objects in scope, it can cover existing and future tagged columns without table-owner-by-table-owner changes.

Why the other options are wrong:

A governed tag policy on pii that limits who is allowed to assign the tag, so that only the governance team decides which columns are masked: Governed-tag assignment permissions control who can label data; they do not mask values for readers. A matching column-mask policy is needed to enforce the requested behavior.

A row filter policy on the catalog that matches the pii tag, which hides the rows containing the tagged values from non-privileged readers: A row filter removes rows based on a predicate. The requirement is to replace tagged column values while preserving the rows, so a column mask is the matching control.

A nightly job that finds the newly tagged columns and runs ALTER COLUMN SET MASK on each one, so the new tables are covered within a day: A nightly per-table alteration creates a delay before new tagged columns are covered and requires ongoing owner-level operations, contrary to defining the rule once.

References:

1. Attribute-based access control in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac>

Question 37

A bundle defines a job named `nightly_load` with a daily schedule. A developer deploys the bundle to the dev target shown below.

```
targets:
  dev:
    mode: development
    default: true
```

What happens to `nightly_load` in the dev workspace?

- A. The job's name gets a [dev username] prefix and its schedule is paused, so the developer's copy does not run on the daily timetable
- B. The job is deployed with its daily schedule active, and development mode only adds a dev tag that makes each of its runs easier to filter
- C. The job is validated but not created, because development mode deploys notebooks and files while leaving the job resources for production use
- D. The job keeps its configured name and its schedule, and development mode only changes which workspace host the bundle is deployed to

Correct answer: A

A. The job's name gets a [dev username] prefix and its schedule is paused, so the developer's copy does not run on the daily timetable

Explanation

Correct answer:

The job's name gets a [dev username] prefix and its schedule is paused, so the developer's copy does not run on the daily timetable

Why this is correct:

The development deployment mode gives each developer a separated copy of resources by prefixing names and pauses schedules and triggers by default. That lets the developer deploy the daily job to dev without creating a second scheduled production-like run.

Why the other options are wrong:

The job keeps its configured name and its schedule, and development mode only changes which workspace host the bundle is deployed to: The schedule remains part of the deployed job configuration; development deployment pauses it rather than removing it.

The job is deployed with its daily schedule active, and development mode only adds a dev tag that makes each of its runs easier to filter: The job does not retain an identical unprefixed name under

the default development behavior. A dev username prefix distinguishes developer resources.

The job is validated but not created, because development mode deploys notebooks and files while leaving the job resources for production use: Unlike a deleted schedule, the deployed trigger remains configured and can be resumed; it is simply paused by the development preset.

References:

1. Declarative Automation Bundles deployment modes:

<https://docs.databricks.com/aws/en/dev-tools/bundles/deployment-modes>

Question 38

Both periods below cover 20 runs of an extract, transform, and publish job on similar input. Duration is the median successful run; all recent failures occurred in transform.

Period	Failed runs	Duration
-----	-----	-----
Earlier	1	12 min
Recent	5	18 min

What should the engineer conclude and check first?

- A. Speed worsened but reliability held steady, since both of the periods had 20 runs; compare each task's recent duration against its earlier baseline
- B. Reliability worsened while speed held steady, since 18 minutes is within the normal variance; review the transform errors and ignore the task durations
- C. Both of the measures regressed, so rerun the recent failures on larger compute and compare their total duration against the earlier median
- D. Reliability and speed both worsened; review transform's recent failures and compare each task's recent duration against its own baseline

Correct answer: D

D. Reliability and speed both worsened; review transform's recent failures and compare each task's recent duration against its own baseline

Explanation

Correct answer:

Reliability and speed both worsened; review transform's recent failures and compare each task's recent duration against its own baseline

Why this is correct:

The newer period has more failures and a longer median successful duration, so both reliability and speed have deteriorated. Since failures are confined to transform, compare that task's failures and duration against its own baseline to localize the regression.

Why the other options are wrong:

Reliability worsened while speed held steady, since 18 minutes is within the normal variance; review the transform errors and ignore the task durations: This reverses both measured trends: failures increased and median duration rose in the recent period, indicating worse reliability and slower successful runs.

Speed worsened but reliability held steady, since both of the periods had 20 runs; compare each

task's recent duration against its earlier baseline: A reliability improvement would require fewer failures, but the recent period has more. The duration also worsened, so attributing the pattern only to an input change overlooks the transform failures.

Both of the measures regressed, so rerun the recent failures on larger compute and compare their total duration against the earlier median: A shorter median duration would show faster successful runs, but the recent median is longer. The failures are also clustered in transform, so a whole-job compute change is not the evidence-led first step.

References:

1. Monitor job runs: <https://docs.databricks.com/aws/en/jobs/monitor>

Question 39

The `audit_readers` group has `USE CATALOG` on `corp` plus `USE SCHEMA` and `SELECT` on `corp.audit`. A new table is then created in `corp.audit`, with no row filters or column masks.

What must be granted before the group can read the new table?

- A. `SELECT` on the new table, because privileges granted on a schema do not extend to tables created after the grant was made
- B. Nothing further, since `SELECT` granted on a schema is inherited by every current and future table within that schema
- C. `BROWSE` on the `corp` catalog, which the group needs so that the new table becomes visible to its members' queries
- D. `MANAGE` on the new table, because schema-level `SELECT` covers only the tables that already existed when that grant was made

Correct answer: B

B. Nothing further, since `SELECT` granted on a schema is inherited by every current and future table within that schema

Explanation

Correct answer:

Nothing further, since `SELECT` granted on a schema is inherited by every current and future table within that schema

Why this is correct:

Unity Catalog privileges granted on a schema are inherited by the schema's current and future child tables. Since the group already has `SELECT`, `USE SCHEMA`, and `USE CATALOG`, a newly created table is readable without a separate table grant.

Why the other options are wrong:

MANAGE on the new table, because schema-level `SELECT` covers only the tables that already existed when that grant was made: `MANAGE` controls administration of the object and is not needed for data reads. The schema-level `SELECT` grant already covers the newly created table.

BROWSE on the `corp` catalog, which the group needs so that the new table becomes visible to its members' queries: `BROWSE` supports discovery and metadata visibility, not reading table contents. The group already has the necessary usage and `SELECT` privileges.

SELECT on the new table, because privileges granted on a schema do not extend to tables created after the grant was made: This overlooks privilege inheritance: schema-level `SELECT` applies to tables created later as well as those present when the privilege was granted.

References:

1. Unity Catalog privileges reference:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/privileges-reference>

Question 40

A Declarative Automation Bundle must define and deploy the Lakeflow Job shown below together with its source files.

```
bundle:
  name: inventory_pipeline
  _____:
    jobs:
      refresh_inventory:
        name: refresh_inventory
        tasks:
          - task_key: refresh
            notebook_task:
              notebook_path: ./src/refresh.py
```

Which top-level key belongs in the blank?

- A.** artifacts, which describes the build outputs, so jobs listed under it are packaged and then uploaded with the other built files
- B.** variables, which declares the reusable values, so jobs placed under it are then substituted wherever the bundle refers to them
- C.** targets, which lists the bundle's deployment environments, so a job defined directly under it is deployed to every environment the bundle declares
- D.** resources, which holds the jobs, the pipelines, and the other workspace objects that the bundle declares and then creates or updates in the target workspace

Correct answer: D

D. resources, which holds the jobs, the pipelines, and the other workspace objects that the bundle declares and then creates or updates in the target workspace

Explanation

Correct answer:

resources, which holds the jobs, the pipelines, and the other workspace objects that the bundle declares and then creates or updates in the target workspace

Why this is correct:

Declarative Automation Bundles place deployable workspace resources, including jobs and pipelines, under the top-level resources mapping. The YAML shown is defining a job object that the bundle should deploy alongside its source files.

Why the other options are wrong:

targets, which lists the bundle's deployment environments, so a job defined directly under it is

deployed to every environment the bundle declares: targets defines deployment contexts such as dev and prod. A job belongs in the resources collection and can then be overridden by a target.

variables, which declares the reusable values, so jobs placed under it are then substituted wherever the bundle refers to them: Reusable values belong in this section; it does not register deployable workspace objects such as a job.

artifacts, which describes the build outputs, so jobs listed under it are packaged and then uploaded with the other built files: Build outputs may be defined here, but the job itself is a workspace resource and would not be declared there.

References:

1. Configuration reference: <https://docs.databricks.com/aws/en/dev-tools/bundles/reference>

Question 41

The orders table has order_id and an items column of type ARRAY<STRING>, and some orders have a NULL or empty array. Silver needs one row per item, with those orders appearing once with a NULL item.

Which query produces those rows?

- A. `SELECT order_id, flatten(items) AS item FROM orders;`
- B. `SELECT order_id, explode(items) AS item FROM orders;`
- C. `SELECT order_id, explode_outer(items) AS item FROM orders;`
- D. `SELECT order_id, posexplode(items) AS (pos, item) FROM orders;`

Correct answer: C

C. `SELECT order_id, explode_outer(items) AS item FROM orders;`

Explanation

Correct answer:

`SELECT order_id, explode_outer(items) AS item FROM orders;`

Why this is correct:

explode_outer emits one row per array element and uses outer semantics to retain the parent row when the array is NULL or empty, producing a NULL item in that case. That gives silver both one-row-per-item behavior and the required placeholder rows.

Why the other options are wrong:

SELECT order_id, explode(items) AS item FROM orders;; explode drops a parent row when its array is NULL or empty. That loses precisely the orders the requirement says must appear once with a NULL item.

SELECT order_id, posexplode(items) AS (pos, item) FROM orders;; inline expands fields from an array of structs into columns. Here the array contains strings, so there are no struct fields to expand, and it is not the outer generator requested.

SELECT order_id, flatten(items) AS item FROM orders;; A left join to another table would not unnest the items array into one row per element. The array itself must be exploded.

References:

1. explode_outer table-valued generator function:

https://docs.databricks.com/aws/en/sql/language-manual/functions/explode_outer

Question 42

A job runs load, then notify_failure, whose Run if condition is At least one failed. After a run in which load succeeded, the Jobs UI shows notify_failure as Excluded and the run as Succeeded.

How should the engineer read this?

- A. As expected, because its Run if requirement was not satisfied, so the task itself never ran and the job run could still pass
- B. As a failure of notify_failure, because Excluded marks a task that had crashed before it could start and that now needs a repair run
- C. As a disabled task, because Excluded means someone had switched notify_failure off in the job settings for this single run
- D. As a concurrency problem, because Excluded means the task had waited for a free run slot and was then dropped when none opened up

Correct answer: A

A. As expected, because its Run if requirement was not satisfied, so the task itself never ran and the job run could still pass

Explanation

Correct answer:

As expected, because its Run if requirement was not satisfied, so the task itself never ran and the job run could still pass

Why this is correct:

A task whose Run if condition is not met is marked Excluded because it was intentionally skipped, not because it failed. With load successful, the failure-only notification task does not execute and the overall job can succeed.

Why the other options are wrong:

As a failure of notify_failure, because Excluded marks a task that had crashed before it could start and that now needs a repair run: Excluded means the task's condition prevented it from running; a crash after starting would be a failed task, not an excluded one.

As a disabled task, because Excluded means someone had switched notify_failure off in the job settings for this single run: A disabled task is configured as disabled. Here the UI state follows from the unsatisfied Run if condition, with no evidence that anyone disabled it.

As a concurrency problem, because Excluded means the task had waited for a free run slot and was then dropped when none opened up: Concurrency controls whether a run starts or waits; it does not explain one downstream task being excluded because its condition was false.

References:

1. Configure task dependencies: <https://docs.databricks.com/gcp/en/jobs/run-if>

Question 43

A BI team runs SQL dashboards on curated Delta tables, with demand spiking each morning and near zero overnight. It wants startup in seconds, extra clusters when queries queue, and no VMs in its own cloud account.

Which compute choice fits best?

- A. A classic SQL warehouse kept at a single cluster, which runs in the classic compute plane and stays on all day to avoid start delays for users
- B. A pro SQL warehouse with multi-cluster scaling, which runs in the workspace's classic compute plane and then adds clusters as queries start to queue up
- C. A serverless SQL warehouse, which runs in the Databricks-managed plane and then adds clusters as concurrent dashboard queries start to queue up
- D. A classic all-purpose cluster with autoscaling and auto-termination, which adds workers when under load and then shuts down during the quiet overnight hours

Correct answer: C

C. A serverless SQL warehouse, which runs in the Databricks-managed plane and then adds clusters as concurrent dashboard queries start to queue up

Explanation

Correct answer:

A serverless SQL warehouse, which runs in the Databricks-managed plane and then adds clusters as concurrent dashboard queries start to queue up

Why this is correct:

Serverless SQL warehouses are designed for BI workloads with demand that changes sharply over the day. They start in seconds, add clusters when queued queries need capacity, and run on Databricks-managed infrastructure, matching all three constraints in the stem.

Why the other options are wrong:

A classic SQL warehouse kept at a single cluster, which runs in the classic compute plane and stays on all day to avoid start delays for users: With its cluster count fixed at one, this warehouse cannot add parallel capacity for the morning spike; classic startup also takes minutes, not seconds.

A pro SQL warehouse with multi-cluster scaling, which runs in the workspace's classic compute plane and then adds clusters as queries start to queue up: Even though a pro warehouse can scale, its compute runs in the customer's cloud account and startup typically takes minutes.

A classic all-purpose cluster with autoscaling and auto-termination, which adds workers when under load and then shuts down during the quiet overnight hours: Autoscaling and auto-termination

reduce idle cost on a notebook cluster, but this remains classic compute that the team must configure and manage; it also lacks the serverless SQL warehouse startup behavior.

References:

1. SQL warehouse types:

<https://docs.databricks.com/aws/en/compute/sql-warehouse/warehouse-types>

Question 44

The masking function `finance.security.mask_account` already exists. It takes a **STRING** and returns a masked **STRING**. An admin must apply it to the `account_number` column of `finance.curated.payments`.

Which statement applies the mask?

- A. `ALTER TABLE finance.curated.payments ALTER COLUMN account_number SET MASK finance.security.mask_account;`
- B. `ALTER TABLE finance.curated.payments SET ROW FILTER finance.security.mask_account ON (account_number);`
- C. `ALTER TABLE finance.curated.payments ALTER COLUMN account_number SET TAGS ('mask' = 'finance.security.mask_account');`
- D. `ALTER FUNCTION finance.security.mask_account SET MASK ON finance.curated.payments.account_number;`

Correct answer: A

A. `ALTER TABLE finance.curated.payments ALTER COLUMN account_number SET MASK finance.security.mask_account;`

Explanation

Correct answer:

`ALTER TABLE finance.curated.payments ALTER COLUMN account_number SET MASK finance.security.mask_account;`

Why this is correct:

A column mask is attached to a specific table column with `ALTER TABLE ... ALTER COLUMN ... SET MASK`. The supplied function accepts and returns a string, matching `account_number`'s type, so this statement applies it at read time.

Why the other options are wrong:

`ALTER TABLE finance.curated.payments SET ROW FILTER finance.security.mask_account ON (account_number);`: A row filter decides which whole rows are visible; it does not replace an individual column's value with a masked string.

`ALTER TABLE finance.curated.payments ALTER COLUMN account_number SET TAGS ('mask' = 'finance.security.mask_account');`: A tag labels an object for governance and policy matching.

Assigning a tag alone does not invoke the masking function or mask query results.

ALTER FUNCTION finance.security.mask_account SET MASK ON

finance.curated.payments.account_number; The operation is issued on the table column, not by altering the function. ALTER FUNCTION does not attach a mask to a table field.

References:

1. Manually apply row filters and column masks:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply>

Question 45

A report must list every warehouse and show available_units only from inventory rows with status active. Warehouses lacking active inventory must still appear, with a null available_units.

```
SELECT w.warehouse_id, i.available_units
FROM warehouses AS w
LEFT JOIN inventory AS i
  ON w.warehouse_id = i.warehouse_id
```

Where should the status condition go?

- A.** In the join condition as `AND i.status = 'active'`, which filters inventory before matching so warehouses without active rows get a null
- B.** In an `INNER JOIN` condition as `AND i.status = 'active'`, which matches only active rows while still returning every warehouse listed
- C.** In a `WHERE i.status = 'active' OR i.status IS NULL` clause, which keeps warehouses with no inventory rows as well as active matches
- D.** In a `WHERE i.status = 'active'` clause after the join, which keeps warehouses without active rows because the join already preserved them

Correct answer: A

- A.** In the join condition as `AND i.status = 'active'`, which filters inventory before matching so warehouses without active rows get a null

Explanation

Correct answer:

In the join condition as `AND i.status = 'active'`, which filters inventory before matching so warehouses without active rows get a null

Why this is correct:

A left join preserves every warehouse only if the active-status restriction is part of matching. Adding the predicate to `ON` lets unmatched warehouses remain, with `NULL` inventory columns; a post-join `WHERE` filter would remove those `NULL`-extended rows.

Why the other options are wrong:

In a `WHERE i.status = 'active'` clause after the join, which keeps warehouses without active rows because the join already preserved them: The `WHERE` predicate rejects the `NULL` status produced for warehouses with no matching inventory, effectively eliminating those warehouses from the left-join result.

In a `WHERE i.status = 'active' OR i.status IS NULL` clause, which keeps warehouses with no

inventory rows as well as active matches: When a warehouse has only inactive inventory matches, the join does not generate a NULL status row; the WHERE clause drops that warehouse instead of preserving it.

In an INNER JOIN condition as AND i.status = 'active', which matches only active rows while still returning every warehouse listed: An inner join discards warehouses without an active match by definition. Putting the status test in its ON clause cannot restore rows that the join type removes.

References:

1. JOIN: <https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-select-join>

Question 46

A pull-request check must catch configuration errors in a Declarative Automation Bundle's dev target and fail on any error, leaving the dev workspace untouched and starting no job.

Which command should the check run?

- A.** databricks bundle run -t dev daily_load, which validates the bundle configuration first and then starts the daily_load job in the dev workspace
- B.** databricks bundle validate -t dev, which loads the configuration, resolves the dev target's settings, and exits with an error if any are found
- C.** databricks bundle summary -t dev, which lists the resources that the bundle has already deployed to the dev target along with their workspace URLs
- D.** databricks bundle deploy -t dev, which stops early on configuration errors and otherwise uploads the bundle files and updates the dev resources

Correct answer: B

B. databricks bundle validate -t dev, which loads the configuration, resolves the dev target's settings, and exits with an error if any are found

Explanation

Correct answer:

databricks bundle validate -t dev, which loads the configuration, resolves the dev target's settings, and exits with an error if any are found

Why this is correct:

bundle validate -t dev checks the bundle configuration against the selected target's settings and reports validation errors. It does not deploy resources or start a workflow, so it fits a pull-request gate that must leave the workspace untouched.

Why the other options are wrong:

databricks bundle summary -t dev, which lists the resources that the bundle has already deployed to the dev target along with their workspace URLs: bundle summary reports the bundle identity and deployed resources; it is not the command that validates the configuration against resource schemas.

databricks bundle deploy -t dev, which stops early on configuration errors and otherwise uploads the bundle files and updates the dev resources: Deploy is a publishing operation: after validation it synchronizes files and creates or updates workspace resources, violating the no-workspace-changes condition.

databricks bundle run -t dev daily_load, which validates the bundle configuration first and then

starts the daily_load job in the dev workspace: bundle run launches the named job after using the dev target. Starting a job is specifically excluded by the requirement.

References:

1. **bundle command group:** <https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands>

Question 47

A job runs extract, transform, and publish in sequence; the first two take 40 minutes. Today publish failed on an expired credential, which is now fixed, and the upstream outputs are still valid.

How should the engineer finish today's run most efficiently?

- A. Run the job now with the same parameters, which reuses the successful task results from today's failed run and retries only the publish task
- B. Repair the run and select all three tasks, so publish uses the fixed credential against the outputs that extract and transform then rebuild
- C. Rerun the transform task along with every task downstream of it, giving the publish task its input from a newly transformed dataset
- D. Repair the run and rerun only the failed publish task, keeping the earlier successful extract and transform results from today's run

Correct answer: D

D. Repair the run and rerun only the failed publish task, keeping the earlier successful extract and transform results from today's run

Explanation

Correct answer:

Repair the run and rerun only the failed publish task, keeping the earlier successful extract and transform results from today's run

Why this is correct:

Repairing a multi-task job reruns unsuccessful tasks and their dependents while reusing successful task results. Since only publish failed and the upstream outputs remain valid, repairing this run avoids spending another 40 minutes on extract and transform.

Why the other options are wrong:

Repair the run and select all three tasks, so publish uses the fixed credential against the outputs that extract and transform then rebuild: Selecting all three tasks deliberately reruns the successful upstream work too, wasting time when those outputs are already valid.

Rerun the transform task along with every task downstream of it, giving the publish task its input from a newly transformed dataset: Rerunning transform and its dependent publish repeats work that succeeded before the credential failure. The repair can target only the failed publish task.

Run the job now with the same parameters, which reuses the successful task results from today's failed run and retries only the publish task: Starting a fresh run executes the job DAG as a new run; it

does not reuse successful task results from the failed run.

References:

1. **Troubleshoot and repair job failures:** <https://docs.databricks.com/gcp/en/jobs/repair-job-failures>

Question 48

A Lakeflow Spark Declarative Pipelines table defines the expectation below, and an incoming record has `distance_km = -4`.

```
CONSTRAINT valid_distance EXPECT (distance_km >= 0)
ON VIOLATION DROP ROW
```

What happens to that record during the update?

- A.** It is left out of the target table, and the pipeline then records that violation in the data quality metrics kept for `valid_distance`
- B.** It stops the whole update run with an error before any of the records from that batch can be committed to the target table
- C.** It is written to the target with `distance_km` set to a `NULL` value, and the violation is counted in the pipeline's data quality metrics
- D.** It is written to the target unchanged, and the pipeline then records that violation in the data quality metrics kept for `valid_distance`

Correct answer: A

- A.** It is left out of the target table, and the pipeline then records that violation in the data quality metrics kept for `valid_distance`

Explanation

Correct answer:

It is left out of the target table, and the pipeline then records that violation in the data quality metrics kept for `valid_distance`

Why this is correct:

An expectation with `ON VIOLATION DROP ROW` excludes a record that fails its predicate from the target table, while recording expectation metrics in the pipeline event log. Since `-4` is below the required zero threshold, the row is dropped and measured as a violation.

Why the other options are wrong:

It is written to the target with `distance_km` set to a `NULL` value, and the violation is counted in the pipeline's data quality metrics: The `DROP ROW` action does not fail the entire update. Pipeline failure is the behavior associated with an expectation configured to fail on violation.

It stops the whole update run with an error before any of the records from that batch can be committed to the target table: The record does not remain in the target: the configured action explicitly drops violating rows. Metrics are recorded, but the negative-distance row is excluded.

It is written to the target unchanged, and the pipeline then records that violation in the data quality metrics kept for valid_distance: EXPECT evaluates the predicate for each row and applies the declared violation action; it does not transform the field's value to zero.

References:

1. Manage data quality with pipeline expectations:

<https://docs.databricks.com/gcp/en/ldp/expectations>

Question 49

In a deliveries DataFrame, fee is NULL for free deliveries, which must count as 0. A report needs exactly one row with the total and the average fee across all deliveries. Assume F is imported.

Which approach produces this result?

- A. Fill the NULL fees with 0 using fillna, then group by fee and aggregate F.sum and F.avg, which then returns the total and average as one row
- B. Aggregate F.sum and F.avg on fee without grouping the rows, because avg already treats each NULL fee as zero when it computes the mean
- C. Aggregate F.sum on F.coalesce of fee and 0 and F.avg on F.coalesce of fee and 0 without grouping the rows, which returns one row where free deliveries count as zero in both calculations
- D. Aggregate F.sum on fee and divide it by F.count on fee without grouping the rows, which averages over every delivery including free ones

Correct answer: C

C. Aggregate F.sum on F.coalesce of fee and 0 and F.avg on F.coalesce of fee and 0 without grouping the rows, which returns one row where free deliveries count as zero in both calculations

Explanation

Correct answer:

Aggregate F.sum on F.coalesce of fee and 0 and F.avg on F.coalesce of fee and 0 without grouping the rows, which returns one row where free deliveries count as zero in both calculations

Why this is correct:

A global aggregation without grouping returns one result row. Coalescing fee to zero before both sum and avg includes free deliveries in each calculation and ensures even an all-free input has a total and average of zero rather than NULL.

Why the other options are wrong:

Aggregate F.sum and F.avg on fee without grouping the rows, because avg already treats each NULL fee as zero when it computes the mean: avg ignores NULL values; it does not count each one as zero. It would calculate the mean only over paid deliveries and fail the stated denominator requirement.

Aggregate F.sum on fee and divide it by F.count on fee without grouping the rows, which averages over every delivery including free ones: count(fee) counts only non-NULL fees, so dividing by it excludes free deliveries from the average and can divide by zero when every delivery is free.

Fill the NULL fees with 0 using fillna, then group by fee and aggregate F.sum and F.avg, which then returns the total and average as one row: Grouping by fee produces separate groups for different

fee values, not the single overall row requested. Filling NULLs is useful, but the grouping changes the output shape.

References:

1. **avg aggregate function:** <https://docs.databricks.com/gcp/en/sql/language-manual/functions/avg>
2. **sum aggregate function:** <https://docs.databricks.com/aws/en/sql/language-manual/functions/sum>

Question 50

An Auto Loader reader uses `cloudFiles.format = binaryFile` with `pathGlobFilter = scan_*.png`. These files arrive together: `scan_12.png`, `scan_12.csv`, `preview_12.png`, and `scan_13.png`. All are readable, and no other filters apply.

Which files does Auto Loader ingest?

- A. Only `scan_12.png` and `scan_13.png`, because a file name must match both the `scan_` prefix and the `.png` extension in the glob pattern
- B. All four of the files, because `binaryFile` ingests every readable file and `pathGlobFilter` applies only to directory names, not to file names
- C. `scan_12.png`, `scan_12.csv`, and `scan_13.png`, because the glob filter admits any file whose file name starts with the `scan_` prefix
- D. `scan_12.png`, `preview_12.png`, and `scan_13.png`, because the glob filter admits any PNG file that lands in the source directory

Correct answer: A

- A. Only `scan_12.png` and `scan_13.png`, because a file name must match both the `scan_` prefix and the `.png` extension in the glob pattern

Explanation

Correct answer:

Only `scan_12.png` and `scan_13.png`, because a file name must match both the `scan_` prefix and the `.png` extension in the glob pattern

Why this is correct:

`pathGlobFilter` filters candidate file names using the glob `scan_*.png`. Both the required prefix and `.png` suffix must match, so only `scan_12.png` and `scan_13.png` qualify; file format does not bypass the filter.

Why the other options are wrong:

All four of the files, because `binaryFile` ingests every readable file and `pathGlobFilter` applies only to directory names, not to file names: `binaryFile` can read arbitrary files as bytes, but discovery filters still apply. Readability alone does not admit names that fail the glob.

`scan_12.png`, `preview_12.png`, and `scan_13.png`, because the glob filter admits any PNG file that lands in the source directory: The `preview_12.png` name has the right extension but does not start with `scan_`, so it does not match the full pattern.

`scan_12.png`, `scan_12.csv`, and `scan_13.png`, because the glob filter admits any file whose file name starts with the `scan_` prefix: A `.csv` filename fails the required suffix even though its prefix matches; both portions of the glob must match.

References:

1. Generic File Source Options:

<https://spark.apache.org/docs/3.5.8/sql-data-sources-generic-options.html>

Question 51

An Auto Loader stream uses an inferred schema with `addNewColumns` mode. A new file adds a column, and the stream stops with an unknown-field error; no existing column changes type.

What has Auto Loader done before stopping, and what comes next?

- A. It has inferred a replacement schema from the latest file alone, so the columns missing from that file then disappear when the stream restarts
- B. It has moved the new field into the rescued data without changing the schema, so the stream resumes on its restart with the same columns
- C. It has kept the old schema as is, so an engineer must add the new column to the schema location by hand before the stream can then restart
- D. It has merged the new column into the schema that is stored at the schema location, so a restart continues with the wider schema

Correct answer: D

D. It has merged the new column into the schema that is stored at the schema location, so a restart continues with the wider schema

Explanation

Correct answer:

It has merged the new column into the schema that is stored at the schema location, so a restart continues with the wider schema

Why this is correct:

In `addNewColumns` mode, Auto Loader merges the newly discovered column into the schema saved at `cloudFiles.schemaLocation`, then fails the current stream with `UnknownFieldException`. A restart loads using that widened schema; existing column types are not changed.

Why the other options are wrong:

It has moved the new field into the rescued data without changing the schema, so the stream resumes on its restart with the same columns: The stream did stop, but Auto Loader has already updated its schema before the exception. The new column is not merely discarded pending manual schema editing.

It has kept the old schema as is, so an engineer must add the new column to the schema location by hand before the stream can then restart: A changed type would be rescued or handled through configured type-widening behavior; the stem says a new column appeared and existing types stayed stable.

It has inferred a replacement schema from the latest file alone, so the columns missing from that

file then disappear when the stream restarts: Auto Loader preserves existing values and schema rather than deleting the new field from the source. The new field is added to the schema store, prompting the restart.

References:

1. Configure schema inference and evolution in Auto Loader:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema>

Question 52

A supported Python notebook runs hourly as a production job. The team must not choose instance types or worker counts, maintain pools or policies, or pay for compute between runs.

Which compute option fits best?

- A. Serverless compute for jobs, which Databricks allocates from its own managed fleet at the very start of each run and then releases once that run finishes
- B. A serverless SQL warehouse attached to the notebook task, which starts up in seconds and then stops when idle between the hourly runs
- C. A classic job cluster drawing from a warm instance pool, which starts up quickly because its idle instances stay ready between the hourly runs
- D. An all-purpose cluster with auto-termination after 10 minutes, which each hourly run then restarts before the notebook code itself begins

Correct answer: A

A. Serverless compute for jobs, which Databricks allocates from its own managed fleet at the very start of each run and then releases once that run finishes

Explanation

Correct answer:

Serverless compute for jobs, which Databricks allocates from its own managed fleet at the very start of each run and then releases once that run finishes

Why this is correct:

Serverless compute for workflows removes cluster infrastructure configuration from the job author and Databricks provisions compute for the run. For a supported notebook workload, compute is allocated as needed and released after the run, addressing both administration and between-run cost concerns.

Why the other options are wrong:

A classic job cluster drawing from a warm instance pool, which starts up quickly because its idle instances stay ready between the hourly runs: A fixed-size classic job cluster still requires selecting instance types and worker capacity, and its lifecycle and configuration remain the team's responsibility.

A serverless SQL warehouse attached to the notebook task, which starts up in seconds and then stops when idle between the hourly runs: Autoscaling adjusts worker counts but does not eliminate choosing compute settings or managing the cluster configuration. It also does not provide the serverless managed lifecycle described.

An all-purpose cluster with auto-termination after 10 minutes, which each hourly run then restarts before the notebook code itself begins: A SQL warehouse is meant for SQL workloads, while the task

is a Python notebook. Pro or classic warehouses also do not provide this workflow-compute model.

References:

1. Run your Lakeflow Jobs with serverless compute for workflows:

<https://docs.databricks.com/aws/en/jobs/run-serverless-jobs>

Question 53

A sales team needs Salesforce objects landed in Unity Catalog tables and kept current as records change. The platform team wants the least ingestion code to own, with API limits and schema changes handled for it.

Which approach fits best?

A. A Lakeflow Connect ingestion pipeline built on its Salesforce connector, which Databricks itself runs and maintains end to end as a managed ingestion service

B. Hourly CSV exports that a Salesforce scheduled job drops into a volume, which Auto Loader then appends to bronze tables with no connector code to maintain

C. A scheduled notebook that calls the Salesforce Bulk API with a modified-timestamp watermark and then merges the changed records into the target tables

D. A Lakehouse Federation connection to Salesforce, managed by Databricks, which then exposes the objects in a foreign catalog for queries run in place

Correct answer: A

A. A Lakeflow Connect ingestion pipeline built on its Salesforce connector, which Databricks itself runs and maintains end to end as a managed ingestion service

Explanation

Correct answer:

A Lakeflow Connect ingestion pipeline built on its Salesforce connector, which Databricks itself runs and maintains end to end as a managed ingestion service

Why this is correct:

Lakeflow Connect's Salesforce ingestion connector creates managed ingestion pipelines that land Salesforce data in Unity Catalog and support incremental updates. The connector removes the need for the team to own API pagination, extraction logic, and the ingestion pipeline code described in the requirement.

Why the other options are wrong:

A scheduled notebook that calls the Salesforce Bulk API with a modified-timestamp watermark and then merges the changed records into the target tables: A custom Bulk API notebook can be built to do incremental merges, but the team would still own pagination, retries, API behavior, and schema handling, which is the maintenance it wants to avoid.

A Lakehouse Federation connection to Salesforce, managed by Databricks, which then exposes the objects in a foreign catalog for queries run in place: Federation exposes supported remote sources for querying in place; it does not copy and maintain the requested current Unity Catalog Delta tables from the Salesforce Platform.

Hourly CSV exports that a Salesforce scheduled job drops into a volume, which Auto Loader then appends to bronze tables with no connector code to maintain: Scheduled exports create a manually maintained file handoff and do not provide managed incremental Salesforce ingestion. Auto Loader only handles files after they have been placed in storage.

References:

1. Salesforce ingestion connector:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/salesforce-concepts>

Question 54

The customers table has customer_id and an address column of type STRUCT<city: STRING, postal_code: STRING>. A query must return city and postal_code as separate columns alongside customer_id.

Which approach produces this output?

- A. Select explode(address), which turns the address struct into one row per field so that city and postal_code appear as separate rows
- B. Select inline(address), which expands the address struct's two fields into separate city and postal_code columns on each row
- C. Select address.city and address.postal_code with dot notation, which returns each address struct field as its own separate top-level column
- D. Select to_json(address), which converts the address struct into one JSON string that holds both the city and postal_code values

Correct answer: C

C. Select address.city and address.postal_code with dot notation, which returns each address struct field as its own separate top-level column

Explanation

Correct answer:

Select address.city and address.postal_code with dot notation, which returns each address struct field as its own separate top-level column

Why this is correct:

A Spark SQL struct exposes its named fields through dot notation. Selecting address.city and address.postal_code projects each nested value as a separate top-level column while retaining the row's customer_id.

Why the other options are wrong:

Select explode(address), which turns the address struct into one row per field so that city and postal_code appear as separate rows: explode is for arrays or maps and produces rows from collection elements. An address struct is a single structured value, not a collection of field rows.

Select inline(address), which expands the address struct's two fields into separate city and postal_code columns on each row: inline expands an array of structs into rows and columns. The address column is one struct rather than an array of structs.

Select to_json(address), which converts the address struct into one JSON string that holds both the city and postal_code values: Serializing the nested value would leave both fields inside one JSON

string rather than returning two typed columns.

References:

1. **STRUCT type:** <https://docs.databricks.com/aws/en/sql/language-manual/data-types/struct-type>

Question 55

A notebook must batch-read the PostgreSQL table `operations.deliveries` through JDBC, with the read itself split across eight parallel connections on the numeric `delivery_id` column. The URL, credentials, and driver are ready.

How should the read be configured?

- A. Set `partitionColumn` to `delivery_id` with `lowerBound`, `upperBound`, and `numPartitions` of 8, so each connection then reads one range of the IDs
- B. Read with the default options and then call `repartition(8)`, which spreads the loaded rows across eight partitions for later processing
- C. Set the `fetchsize` to 8, which tells the JDBC driver to pull the rows through eight parallel connections in batches from the source table
- D. Set `numPartitions` to 8 on its own, which makes Spark open eight separate JDBC connections and divide the table's rows evenly among them

Correct answer: A

A. Set `partitionColumn` to `delivery_id` with `lowerBound`, `upperBound`, and `numPartitions` of 8, so each connection then reads one range of the IDs

Explanation

Correct answer:

Set `partitionColumn` to `delivery_id` with `lowerBound`, `upperBound`, and `numPartitions` of 8, so each connection then reads one range of the IDs

Why this is correct:

JDBC parallel reads use a numeric `partitionColumn` plus `lowerBound`, `upperBound`, and `numPartitions` to split the source read into range queries. Setting eight partitions this way opens parallel read partitions; a later Spark `repartition` cannot parallelize the already completed JDBC transfer.

Why the other options are wrong:

Set `numPartitions` to 8 on its own, which makes Spark open eight separate JDBC connections and divide the table's rows evenly among them: `numPartitions` alone gives Spark no column or value range on which to divide the JDBC query. The partitioning options work together to define the parallel read.

Read with the default options and then call `repartition(8)`, which spreads the loaded rows across eight partitions for later processing: `repartition(8)` runs after the JDBC read has brought data into Spark. It can distribute downstream processing, but it cannot create eight concurrent source connections.

Set the `fetchsize` to 8, which tells the JDBC driver to pull the rows through eight parallel

connections in batches from the source table: The JDBC fetch size sets rows per network round trip; it does not control how many connections read distinct ranges concurrently.

References:

1. JDBC connection: <https://docs.databricks.com/aws/en/connect/jdbc-connection>